

- 1. Show all records from the Customers table.**
- 2. Display only customer names and cities.**
- 3. Find customers from Delhi.**
- 4. Find products with price greater than 1000.**
- 5. Display unique cities from the customer table.**
- 6. Show customers whose names start with 'A'.**
- 7. Count the total number of customers.**
- 8. Find the maximum, minimum, and average salary.**

```
CREATE DATABASE SHOP;
```

```
USE SHOP;
```

```
CREATE TABLE CUSTOMER
```

```
(
```

```
EMPID INT PRIMARY KEY,
```

```
NAME VARCHAR(50),
```

```
CITY VARCHAR(50),
```

```
PRODUCT VARCHAR(50),
```

```
PRICE INT,
```

```
REGION VARCHAR(50),
```

```
CATEGORY VARCHAR(50)
```

```
);
```

```
INSERT INTO CUSTOMER
```

```
VALUES
```

```
(1, "AYUSH", "DELHI", "LAPTOP", 40000, "INDIA", "ELECTORNICS"),
```

```
(2, "RAHUL", "PUNE", "PHONE", 30000, "INDIA", "ELECTRONICS"),
```

```
(3, "DAVID", "NEW YARK", "SHIRT", 500, "USA", "CLOTH"),
```

(4, "SMITH", "MELBORN", "PIZZA", 150, "AUSTRALIA", "FOOD"),
(5, "HEMANT", "DELHI", "RICE", 769, "INDIA", "GROCERY"),
(6, "CAMERON", "LONDON", "BAT", 6479, "ENGLAND", "EQUIPMENT"),
(7, "DEVILERS", "DELHI", "LOCK-KEY", "70", "CAPE TOWN", "HARDWARE");

SELECT * FROM CUSTOMER;

SELECT NAME,CITY FROM CUSTOMER;

SELECT * FROM CUSTOMER WHERE CITY = "DELHI";

SELECT * FROM CUSTOMER WHERE PRICE > 1000;

select distinct CITY FROM CUSTOMER;

select * FROM CUSTOMER WHERE NAME LIKE 'A%';

select count(*) AS TOTAL_CUSTOMER
FROM CUSTOMER ;

SELECT MAX(PRICE) AS MAX_PRICE
FROM CUSTOMER;

SELECT MIN(PRICE) AS MIN_PRICE
FROM CUSTOMER;

SELECT ROUND(avg(PRICE), 2) AS AVERAGE_PRICE
FROM CUSTOMER;

GROUP BY & Aggregation

- 9. Find total sales by region.**
- 10. Find average sales by category.**
- 11. Count customers in each city.**
- 12. Find the category with the highest sales.**
- 13. Find the region with the lowest profit.**
- 14. Show categories having total sales greater than 50,000.**

```
SELECT REGION, SUM(PRICE) AS TOTAL_SALES  
FROM CUSTOMER  
group by(REGION);
```

```
SELECT CATEGORY, ROUND(AVG(PRICE),2) AS AVERAGE_SALES  
FROM CUSTOMER  
group by CATEGORY;
```

```
SELECT CITY, COUNT(EMPID) AS COUNT_CUSTOMER  
FROM CUSTOMER  
GROUP BY CITY;
```

```
SELECT CATEGORY, MAX(PRICE) AS MAX_PRICE  
FROM CUSTOMER  
GROUP BY CATEGORY  
order by MAX_PRICE DESC  
LIMIT 1;
```

```
SELECT REGION , MIN(PRICE) AS MIN_PRICE
```

```
FROM CUSTOMER  
GROUP BY REGION  
order by MIN_PRICE ASC  
LIMIT 1;
```

```
SELECT CITY, SUM(PRICE) AS TOTAL_SALES  
FROM CUSTOMER  
GROUP BY CITY  
having SUM(PRICE) > 50000;
```

joins

- 15. Display customer names along with their order amounts.**
- 16. Find customers who have never placed an order.**
- 17. Find total order amount for each customer.**
- 18. Display all customers and orders using LEFT JOIN**

```
CREATE TABLE ORDERS  
(  
ORDERID INT,  
EMPID INT,  
AMOUNT INT,  
foreign key (EMPID) REFERENCES CUSTOMER(EMPID)  
);
```

```
INSERT INTO ORDERS  
VALUES  
(101, 1, 500),  
(102, 2, 600),  
(103, 3, 800),
```

```
(104, 4, 1000),  
(105, 5, 900),  
(106, 6, 10000),  
(107, 7, 5000);
```

```
SELECT * FROM ORDERS;
```

```
SELECT C.NAME, O.AMOUNT  
FROM CUSTOMER C  
left join ORDERS O ON C.EMPID = O.EMPID;
```

```
SELECT C.CITY, SUM(O.AMOUNT) AS TOTAL_AMOUNT  
FROM CUSTOMER C  
left JOIN ORDERS O ON C.EMPID = O.EMPID  
group by CITY  
order by TOTAL_AMOUNT desc;
```

```
SELECT C.NAME , SUM(O.AMOUNT) AS TOTAL_AMOUNT  
FROM CUSTOMER C  
LEFT JOIN ORDERS O ON C.EMPID = O.EMPID  
group by NAME  
ORDER BY TOTAL_AMOUNT ASC;
```

```
SELECT * FROM CUSTOMER C  
LEFT JOIN ORDERS O ON C.EMPID = O.EMPID;
```

Subqueries

19. Find employees earning more than the average salary.

20. Find the second highest salary.

21. Find the product with the highest sales.

22. Find customers whose sales are above average sales.

```
SELECT * FROM CUSTOMER  
WHERE PRICE > (SELECT avg(PRICE) FROM CUSTOMER);
```

```
SELECT MAX(PRICE)  
FROM CUSTOMER  
WHERE PRICE < (SELECT MAX(PRICE)  
FROM CUSTOMER);
```

```
SELECT PRODUCT  
FROM CUSTOMER  
WHERE PRICE = (SELECT MAX(PRICE)  
FROM CUSTOMER );
```

```
SELECT NAME, PRICE FROM CUSTOMER  
WHERE PRICE > (SELECT AVG(PRICE)  
FROM CUSTOMER);
```